

# IEORE4004

# Optimization Models and Methods

Yaren Bilge Kaya, PhD

Columbia University  
Department of Industrial Engineering and Operations Research



# Outline

---

- Introduction to Network Models
  - Basic concepts: graphs, edges, arcs, trees, ...
- Recap of Traveling Salesperson Problem
  - LP formulation
  - MTZ formulation
- Shortest Path Problem
  - LP formulation
  - Dijkstra's algorithm
- Introduction to NetworkX library

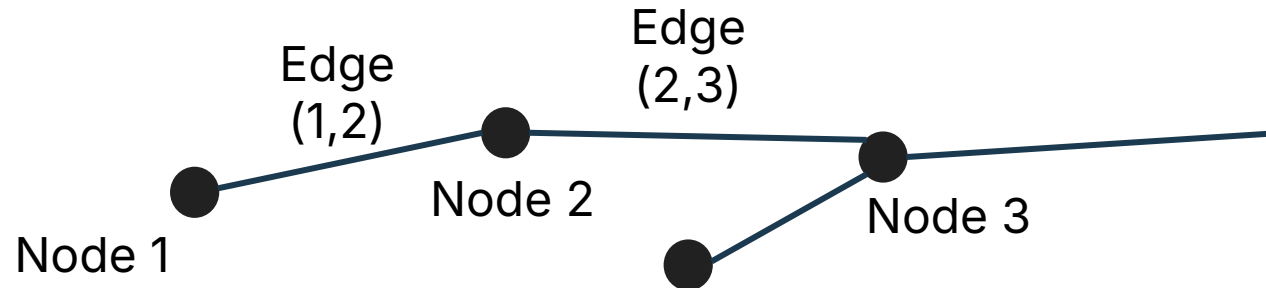
# Introduction to Network Models

- Basic Concepts: Graphs, Edges, Arcs, Trees

# Introduction To Network Models

A graph or network consists of:

- A set  $V$  of vertices (*nodes, points*), and
- A set  $E$  of edges (*arcs, lines*) involving two vertices, each.



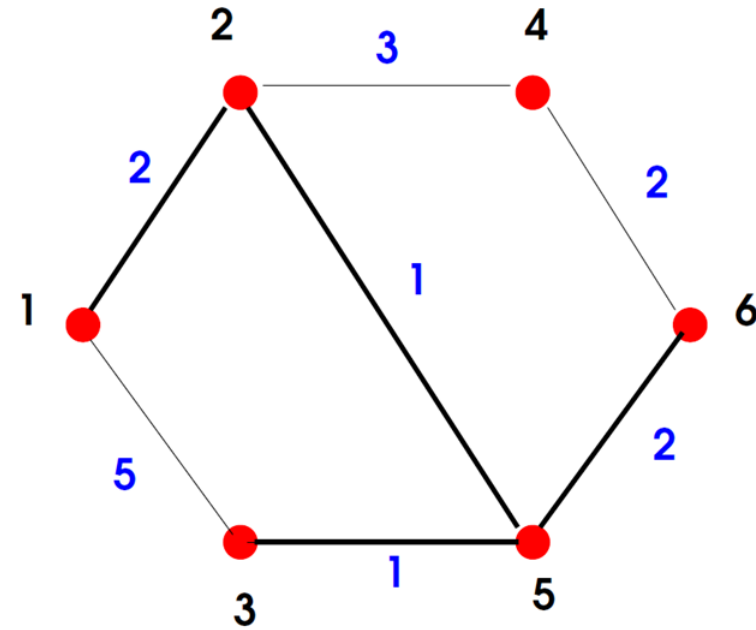
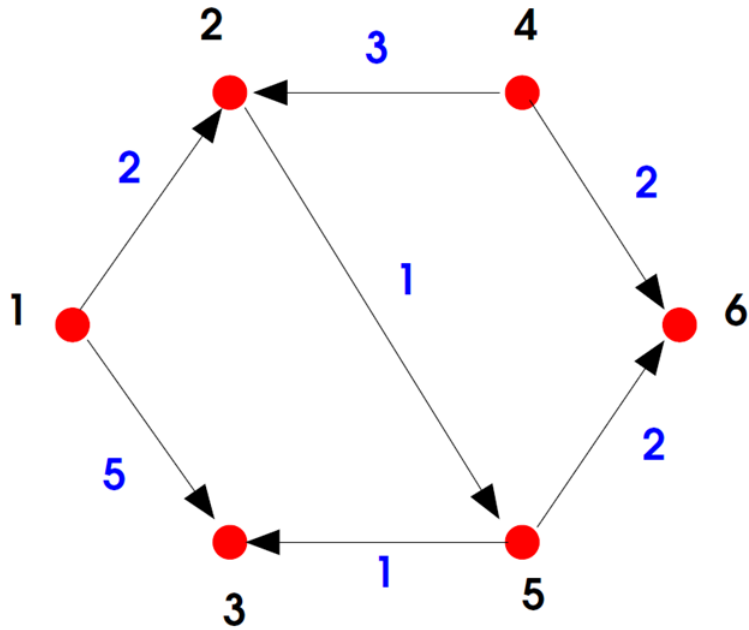
We may write  $V(G)$  to refer to the vertices or  $E(G)$  to refer to the edges.

In the example shown below,

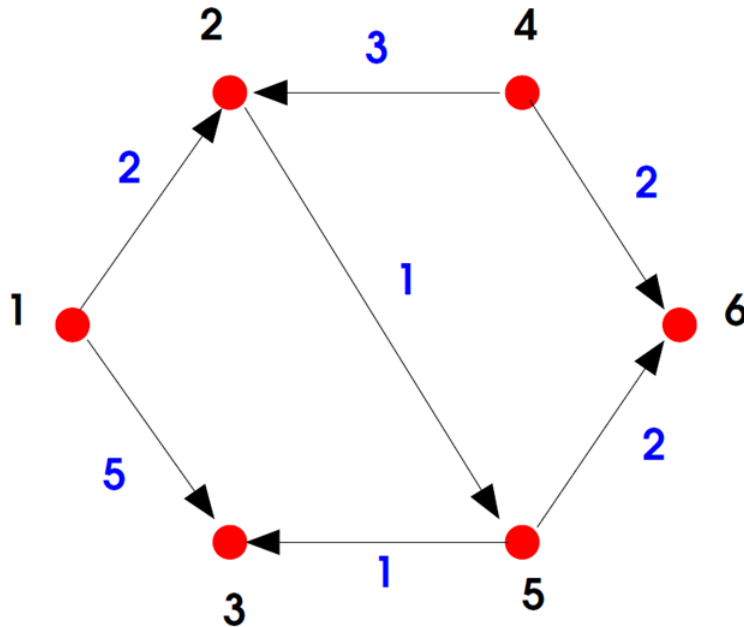
- vertices  $V = \{1,2,3,4,5\}$  and edges  $E = \{(1,2),(2,3),(3,5),(3,4)\}$



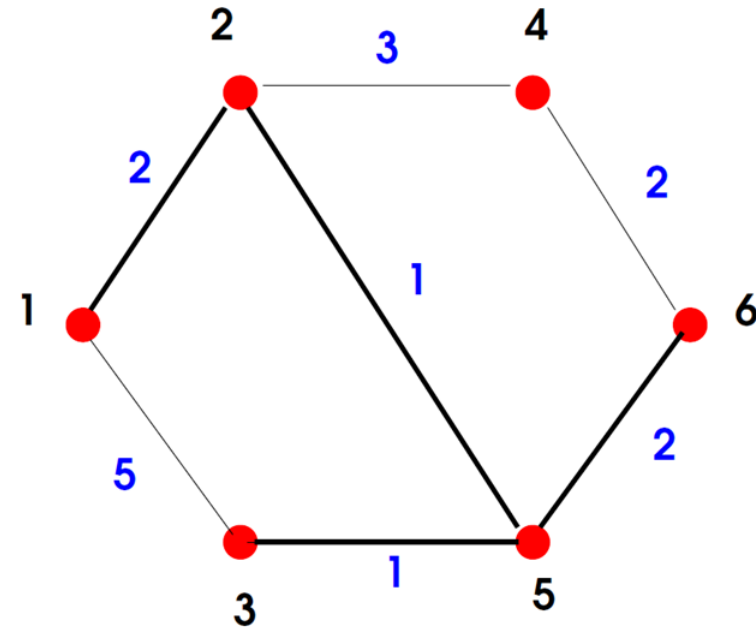
# Directed VS Undirected Networks



# Directed VS Undirected Networks



**Directed**



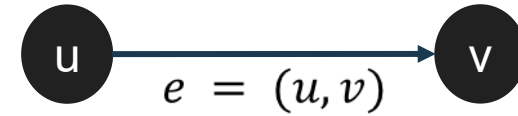
**Undirected**

If an edge  $e$  does not have a direction,  
we treat it the same way as having both directions!

# Directed VS Undirected Networks

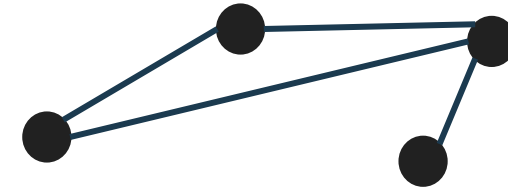
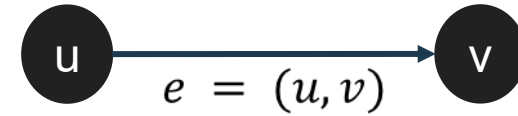
---

- If an edge  $e$  between  $u$  and  $v$  goes from  $u$  to  $v$ , we write  $e = (u, v)$  (or from  $v$  to  $u$ , we write  $e = (v, u)$ )



# Directed VS Undirected Networks

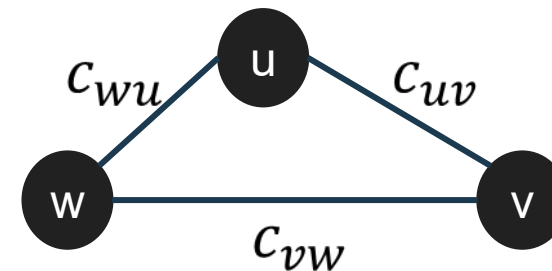
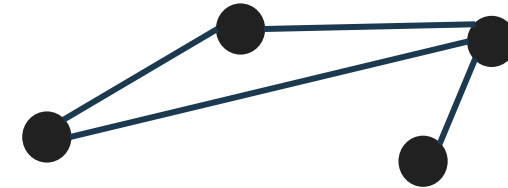
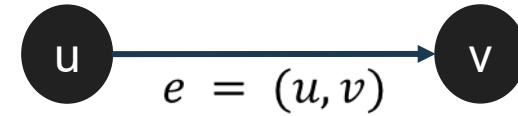
- If an edge  $e$  between  $u$  and  $v$  goes from  $u$  to  $v$ , we write  $e = (u, v)$  (or from  $v$  to  $u$ , we write  $e = (v, u)$ )
- If all edges do not have a direction (are undirected), we say that the **network is undirected**





# Directed VS Undirected Networks

- If an edge  $e$  between  $u$  and  $v$  goes from  $u$  to  $v$ , we write  $e = (u, v)$  (or from  $v$  to  $u$ , we write  $e = (v, u)$ )
- If all edges do not have a direction (are undirected), we say that the **network is undirected**
- Edges may have weight: a weight of edge  $e = (u, v)$  is a real number denoted  $c(e)$  or  $c(u, v)$ ,  $c_e$ ,  $c_{uv}$



# Basic Concepts Of Network Models

---

A sequence of nodes and edges  $v_1, e_1, v_2, e_2, \dots, v_{k-1}, e_k, v_k$  is

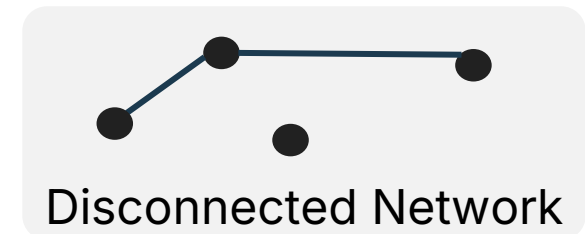
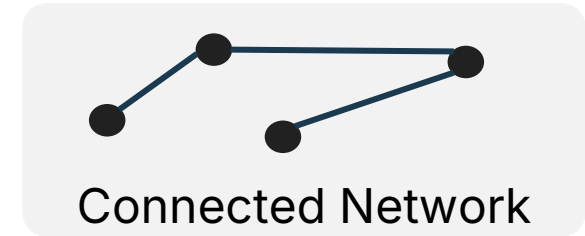
- a path (directed path) if each  $e_i$  goes from  $v_i$  to  $v_{i+1}$
- a chain (undirected path) if each  $e_i$  connects  $v_i$  and  $v_{i+1}$  (in some direction)

Often, we define paths as:

- $e_1, e_2, \dots, e_k$  (we omit vertices), or
- $v_1, v_2, \dots, v_k$  (we omit edges)

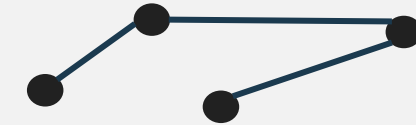
# Basic Concepts Of Network Models

- A **network** is connected if for every two nodes there is a path connecting them; otherwise, it is disconnected

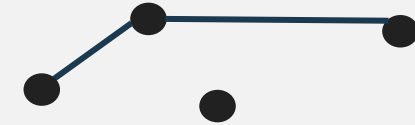


# Basic Concepts Of Network Models

- A **network** is connected if for every two nodes there is a path connecting them; otherwise, it is disconnected

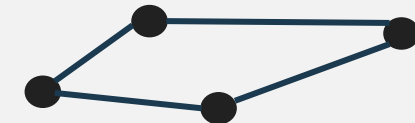


Connected Network



Disconnected Network

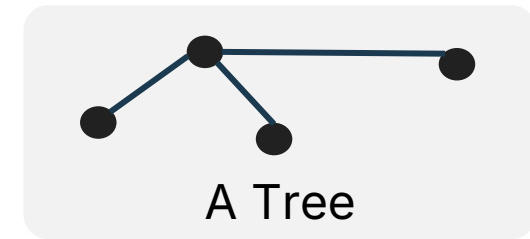
- A **cycle** (loop, circuit) is a path starting and ending in the same node, never repeating any node or edge



Cycle

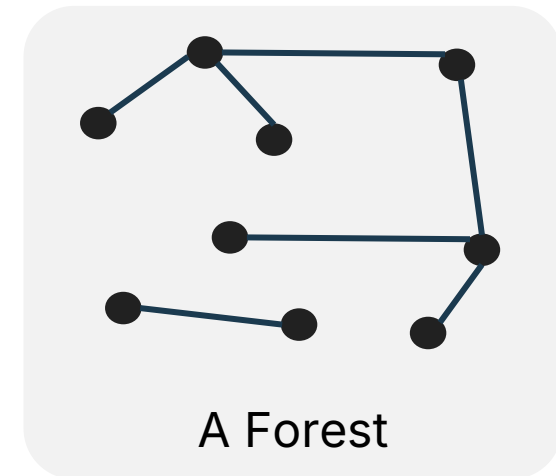
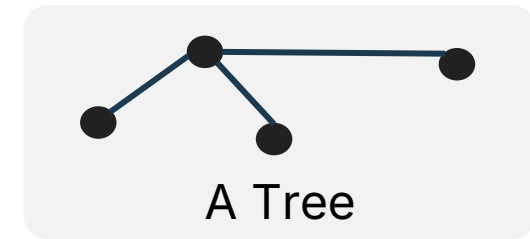
# Basic Concepts Of Network Models

- A **tree** is an undirected graph that is connected and acyclic.
  - In simpler terms, it's a collection of nodes and edges such that there is exactly one path between any two nodes (no loops exist).
  - A tree with  $n$  nodes contains exactly  $n-1$  edges.



# Basic Concepts Of Network Models

- A **tree** is an undirected graph that is connected and acyclic.
  - In simpler terms, it's a collection of nodes and edges such that there is exactly one path between any two nodes (no loops exist).
  - A tree with  $n$  nodes contains exactly  $n-1$  edges.
- A **forest** is an undirected, acyclic graph that contains no cycles (not necessarily connected)



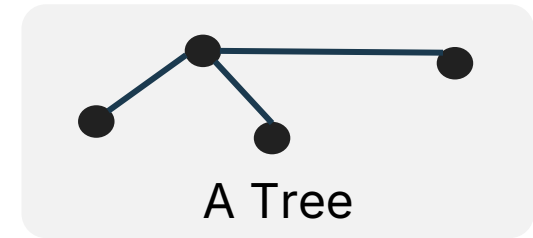


# Basic Concepts Of Network Models

---

Let's evaluate:

1. Adding any edge to a tree creates a cycle.

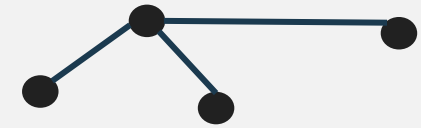


# Basic Concepts Of Network Models

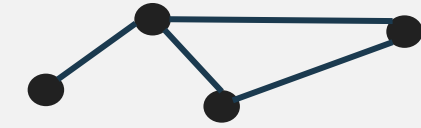
Let's evaluate:

1. Adding any edge to a tree creates a cycle.

✓ **Yes!**



A Tree



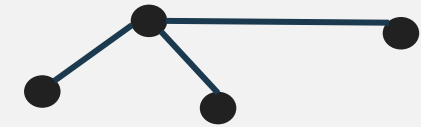
Adding an Edge

# Basic Concepts Of Network Models

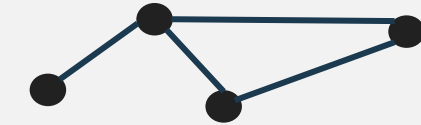
Let's evaluate:

1. Adding any edge to a tree creates a cycle.

✓ **Yes!**



A Tree



Adding an Edge

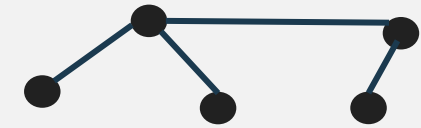
1. Removing any edge from a tree creates a disconnected forest.

# Basic Concepts Of Network Models

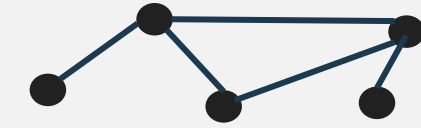
Let's evaluate:

1. Adding any edge to a tree creates a cycle.

✓ Yes!



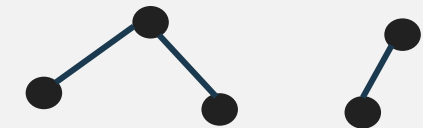
A Tree



Adding an Edge

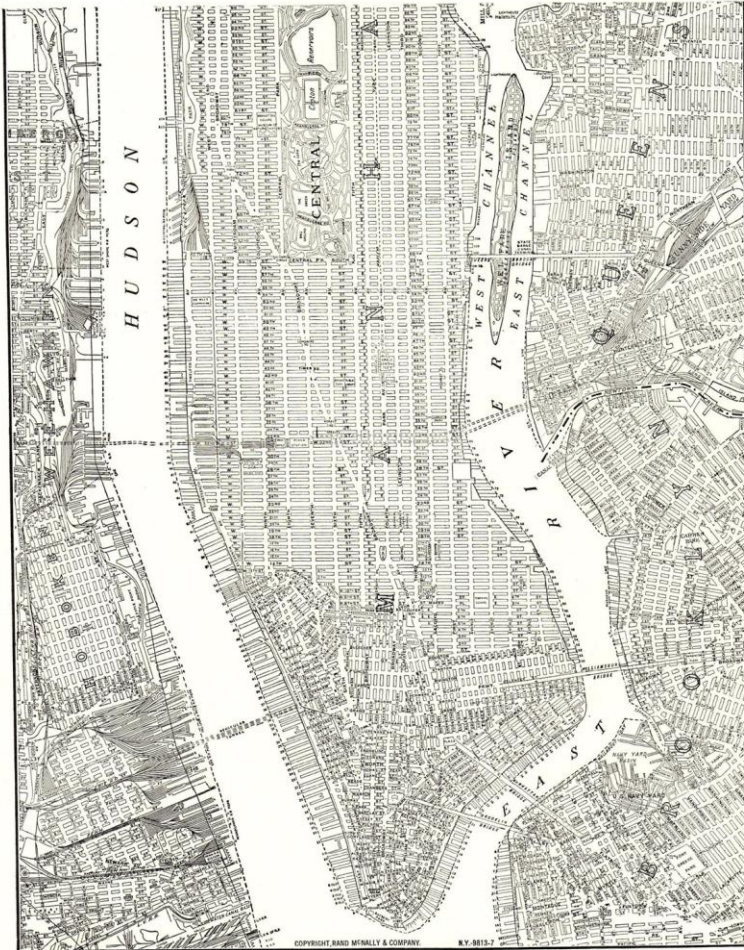
1. Removing any edge from a tree creates a disconnected forest.

✓ Yes!



Removing an Edge

# Examples: Transportation Networks



MAP OF LOWER MANHATTAN, AN ISLAND IN NEW YORK BAY BOUNDED BY THE HUDSON, THE EAST, AND THE HARLEM RIVERS, AND COMPRISING THE BOROUGH OF MANHATTAN OF GREATER NEW YORK. NEW YORK IS THE MOST IMPORTANT PORT ON THE ATLANTIC SEABOARD, AND IS LINED BY GREAT PIERS

**Credit:** 1940 Genuine Antique Map Lower Manhattan (New York). Courtesy of Maps of Antiquity. Retrieved from <https://mapsofantiquity.com>.

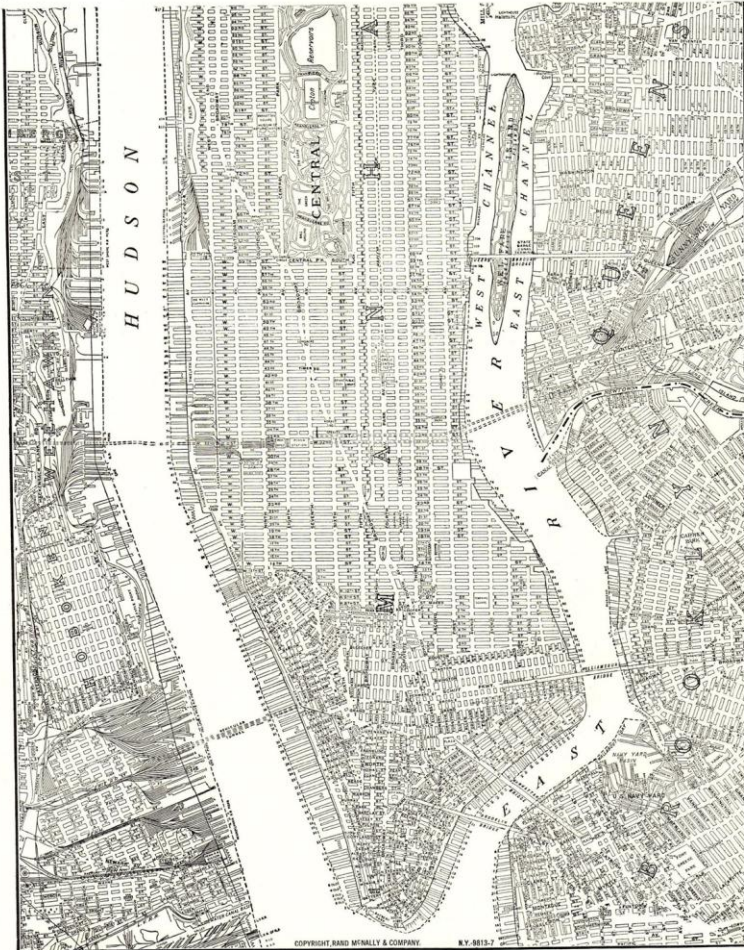
Street networks where nodes are the intersections and edges are the streets



**Credit:** What Life Is Like in the World's Most Crowded Cities. Courtesy of Business Insider. Retrieved from <https://www.businessinsider.com>.



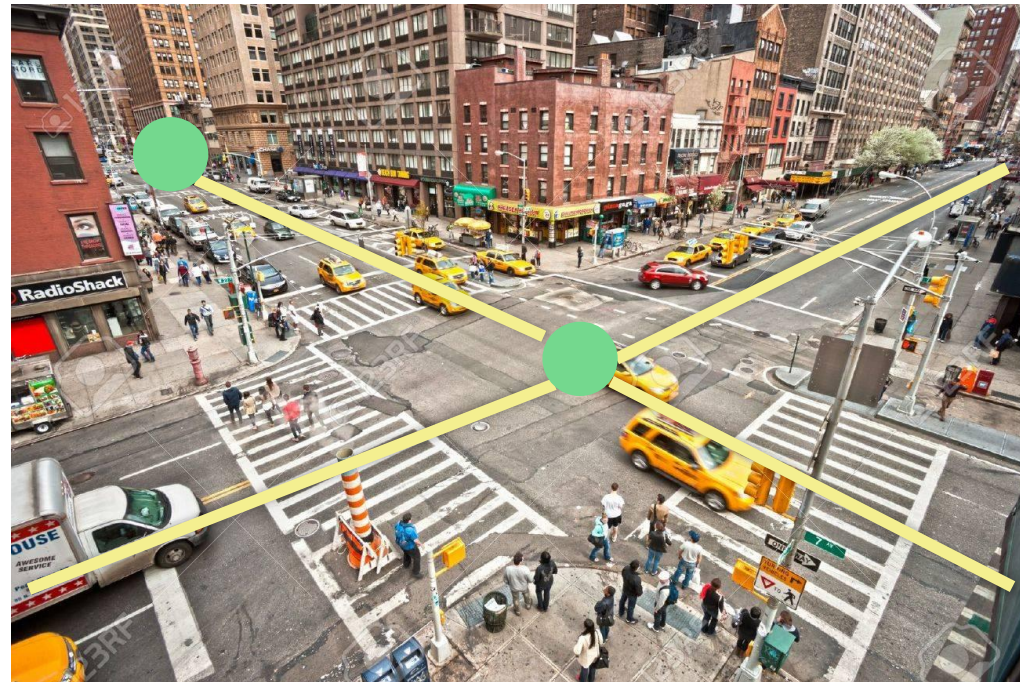
# Examples: Transportation Networks



MAP OF LOWER MANHATTAN, AN ISLAND IN NEW YORK BAY BOUNDED BY THE HUDSON, THE EAST, AND THE HARLEM RIVERS, AND COMPRISING THE BOROUGH OF MANHATTAN OF GREATER NEW YORK. NEW YORK IS THE MOST IMPORTANT PORT ON THE ATLANTIC SEABOARD, AND IS LINED BY GREAT PIERS

**Credit:** 1940 Genuine Antique Map Lower Manhattan (New York). Courtesy of Maps of Antiquity. Retrieved from <https://mapsofantiquity.com>.

Street networks where nodes are the intersections and edges are the streets



**Credit:** What Life Is Like in the World's Most Crowded Cities. Courtesy of Business Insider. Retrieved from <https://www.businessinsider.com>.

# Examples: Social Networks



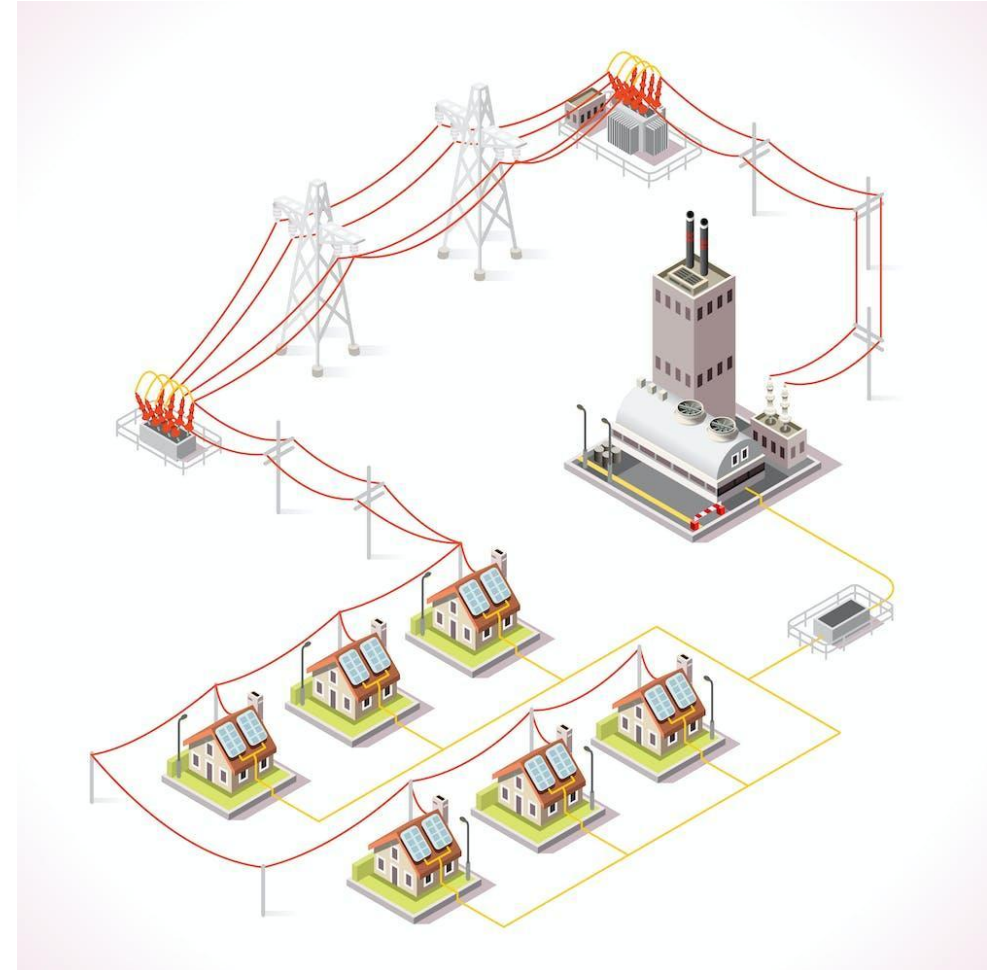
Connections between people, such as friendships, familial ties, and professional relationships.

Websites like LinkedIn and Facebook are online platforms for such networks.



# Examples: Electricity Distribution Networks

- Interconnected power stations, transformers, and transmission lines distributing electricity.

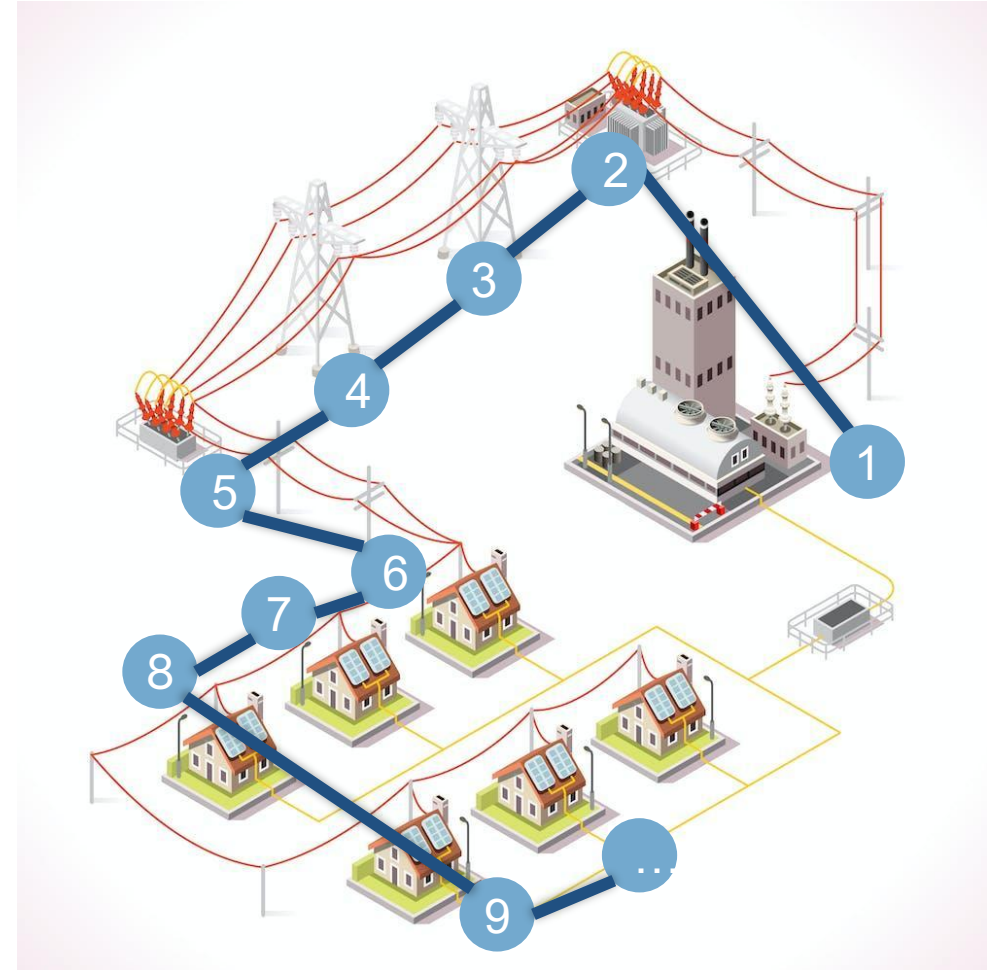


**Credit:** Clean Energy Distribution Chain Infographic Concept, Isometric 3D Electricity Grid Elements Windmill Turbine Power Grid Powerhouse Transformer Providing Electricity Supply to the City Buildings — Vector by aurielaki via Depositphotos.



# Examples: Electricity Distribution Networks

- Interconnected power stations, transformers, and transmission lines distributing electricity.



**Credit:** Clean Energy Distribution Chain Infographic Concept, Isometric 3D Electricity Grid Elements Windmill Turbine Power Grid Powerhouse Transformer Providing Electricity Supply to the City Buildings — Vector by aurielaki via Depositphotos.

# More Examples

---

## Economic Networks

- **Trade Networks:** Connections between countries, cities, or entities based on trade relationships and routes.
- **Financial Networks:** Banks, institutions, and individuals interconnected through loans, deposits, and other financial instruments.

## Biological Networks

- **Neural Networks:** Interconnected neurons in a brain or nervous system.
- **Metabolic Networks:** Enzymes and metabolites in a biological system, representing how various biochemical processes interact.
- **Ecological Networks:** Predators and prey, symbiotic relationships, and other interactions among species in an ecosystem.

# **Introduction to NetworkX Library**

# What is NetworkX?

---

- NetworkX is a Python package to create, manipulate, and study complex networks
- Useful for modeling optimization problems: shortest path, flows, connectivity
- Works with both undirected and directed (graph and digraph) networks
- Integrates well with NumPy, matplotlib for analyses and visualization

# What Can NetworkX Do?

---

- Build graphs (nodes & edges) programmatically
- Visualize networks with labels and custom colors
- Analyze properties: shortest paths, connected components, cycles
- Model real-world optimization problems (transport, flows, logistics)
- Combine with solvers or OR libraries (like Gurobi, PuLP, SciPy)

# NetworkX – Getting Started

---

Now, let's switch to our notebook and see how to use **NetworkX**!



# The Traveling Salesperson Problem

- IP Formulation and the Problem of Subtour Growth
- Miller, Tucker, Zemlin Formulation

# Traveling Salesperson Problem

- A salesperson needs to design a route (“tour”) through  $n$  cities, returning to their original location
  - Would like to do so in the shortest possible distance

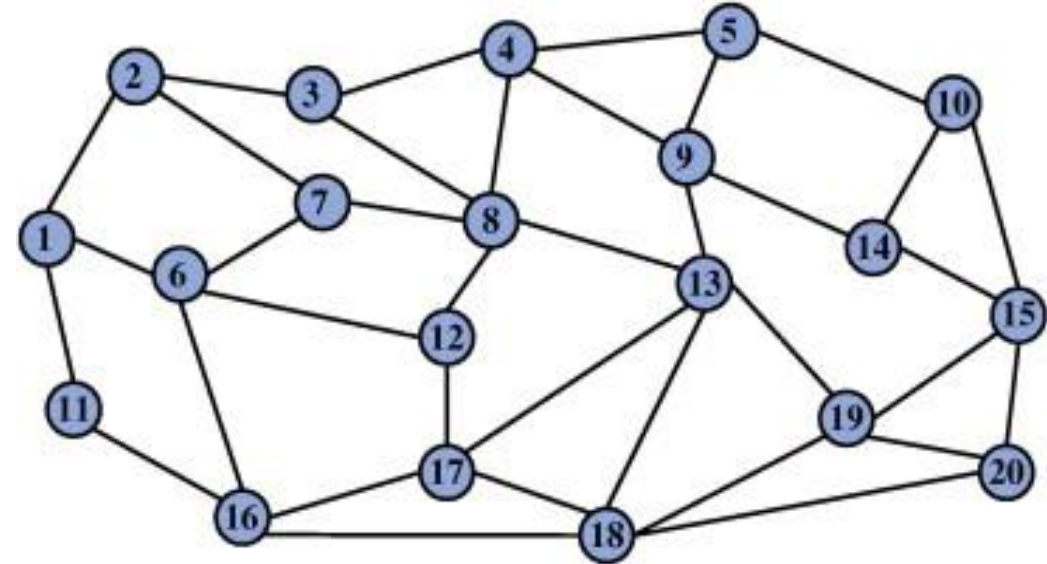


Figure. Structure of the emergency logistics network. Adapted from “Route selection for Emergency Logistics Management: A bio-inspired algorithm,” by X. Zhang, Z. Zhang, Y. Zhang, D. Wei, and Y. Deng, 2013, *Safety Science*, 54, p. 89 (<https://doi.org/10.1016/j.ssci.2012.12.003>). Copyright 2013 by Elsevier.

# Formulating The TSP (Symmetric)

- Suppose  $N$  is a set of cities, indexed by  $i$  and  $j$ .
- Let  $d_{ij}$  be the distance from city  $i$  to city  $j$ ,  $i \neq j$ .
- Let  $x_{ij} = \begin{cases} 1 & \text{if arc from node } i \text{ to node } j \text{ is traversed, } i \neq j; \\ 0 & \text{otherwise.} \end{cases}$

$$\begin{aligned} & \min \quad \sum_{i \in N} \sum_{j \in N: j \neq i} d_{ij} x_{ij} \\ & \text{subject to} \quad \sum_{i \in N: i \neq j} x_{ij} = 1 \quad \forall j \in N, \\ & \quad \sum_{j \in N: j \neq i} x_{ij} = 1 \quad \forall i \in N, \\ & \quad \sum_{i \in S} \sum_{j \in S; j \neq i} x_{ij} \leq |S| - 1, \quad \forall S \subset N \\ & \quad x_{ij} \in \{0, 1\} \quad \forall i \in N, \quad \forall j \in N. \end{aligned}$$

# Miller-Tucker-Zemlin TSP Formulation (Symmetric)

$$\begin{aligned} \min \quad & \sum_{i \in N} \sum_{j \in N} d_{ij} x_{ij} \\ \text{subject to} \quad & \sum_{j \in N \setminus \{i\}} x_{ij} = 1, \quad \forall i \in N \\ & \sum_{j \in N \setminus \{i\}} x_{ji} = 1, \quad \forall i \in N \\ & u_i + 1 \leq u_j + (n - 1)(1 - x_{ij}), \quad \forall i \in N, j \in N, i \neq j \neq 1 \\ & x_{ii} = 0, \quad \forall i \in N \\ & x_{ij} \in \{0, 1\}, \quad \forall i \in N, j \in N, i \neq j \\ & u_1 = 1, \\ & 2 \leq u_i \leq n, \quad i \neq 1. \end{aligned}$$

Usually, the starting city (say city 1) is fixed with  $u_1 = 1$  to anchor the tour and provide a reference point for the ordering.

# The Shortest Path Problem

- LP Formulation
- Dijkstra's algorithm

# Shortest Path (SP) Problem

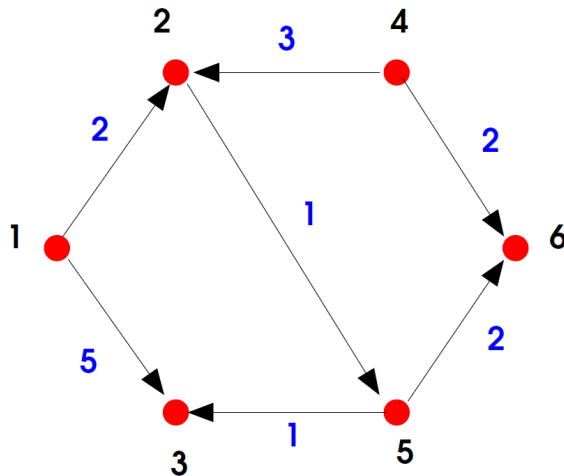
---

Given a network  $G = (V, E)$  with two distinguished vertices  $s, t \in V$ , find a shortest path from  $s$  to  $t$ .

# Shortest Path (SP) Problem

Given a network  $G = (V, E)$  with two distinguished vertices  $s, t \in V$ , find a shortest path from  $s$  to  $t$ .

**Example:** A shortest path from  $s = 1$  to  $t = 6$  is 1, 2, 5, 6 of total length 5, while for  $t = 3$  a shortest path is 1, 2, 5, 3 of length 4.

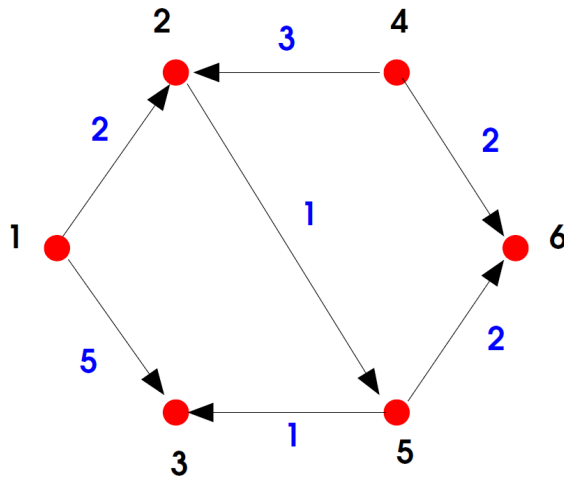




# Shortest Path (SP) Problem

Given a network  $G = (V, E)$  with two distinguished vertices  $s, t \in V$ , find a shortest path from  $s$  to  $t$ .

**Example:** A shortest path from  $s = 1$  to  $t = 6$  is 1, 2, 5, 6 of total length 5, while for  $t = 3$  a shortest path is 1, 2, 5, 3 of length 4.



There is no path from  $s$  to  $t = 4$ ; we indicate this by defining the distance to 4 as  $\infty$ .

---

# **LP Formulation for SP Problem**

# LP Formulation For SP Problem

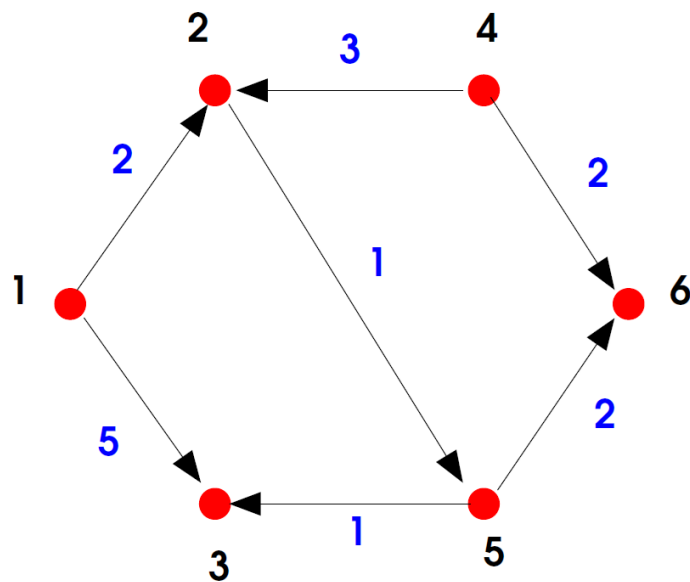
- Decision variables:  $x_{ij}$  for each  $(i, j) \in E$

$$\begin{aligned} \text{Min} \quad & \sum_{(i,j) \in E} w_{ij} x_{ij} \\ & \sum_{j:(i,j) \in E} x_{ij} - \sum_{j:(j,i) \in E} x_{ji} = \begin{cases} 1 & \text{if } i = s \\ -1 & \text{if } i = t \\ 0 & \text{otherwise} \end{cases} \quad \text{for each } i \in V \\ & 0 \leq x_{ij} \quad \text{for each } (i, j) \in E \end{aligned}$$

This formulation is correct if no cycles with negative length exist!

# Example: LP Formulation for SP Problem

Consider the directed network shown below. The problem of finding a shortest path from node 1 to node 6 can be written as follows:



$$\begin{array}{ll} \min & 2x_{12} + 5x_{13} + x_{25} + 3x_{42} + 2x_{46} + x_{53} + 2x_{56} \\ \text{Subject to} & \\ b1 : & x_{12} + x_{13} = 1 \\ b2 : & x_{25} - x_{12} - x_{42} = 0 \\ b3 : & -x_{13} - x_{53} = 0 \\ b4 : & x_{42} + x_{46} = 0 \\ b5 : & x_{53} + x_{56} - x_{25} = 0 \\ b6 : & -x_{46} - x_{56} = -1 \\ & x \geq 0 \end{array}$$

# Car Replacement Problem

---

A new car costs \$12,000. Annual maintenance costs are as follows:  $m_1 = \$2,000$  first year,  $m_2 = \$4,000$  second year,  $m_3 = \$5,000$  third year,  $m_4 = \$9,000$  fourth year, and  $m_5 = \$12,000$  fifth year and on. The car can be sold for  $s_1 = \$7,000$  in the first year, for  $s_2 = \$6,000$  in the second year, for  $s_3 = \$2,000$  in the third year, and for  $s_4 = \$1,000$  in the fourth year of ownership. An existing car can be sold at any time and another new car purchased at \$12,000.

# Car Replacement Problem

A new car costs \$12,000. Annual maintenance costs are as follows:  $m_1 = \$2,000$  first year,  $m_2 = \$4,000$  second year,  $m_3 = \$5,000$  third year,  $m_4 = \$9,000$  fourth year, and  $m_5 = \$12,000$  fifth year and on.

The car can be sold for  $s_1 = \$7,000$  in the first year, for  $s_2 = \$6,000$  in the second year, for  $s_3 = \$2,000$  in the third year, and for  $s_4 = \$1,000$  in the fourth year of ownership. An existing car can be sold at any time and another new car purchased at \$12,000.



What buying/selling strategy for the next 5 years minimizes the total cost of ownership?

# Car Replacement Problem

---

- Nodes =  $\{1, 2, 3, 4, 5\}$
- Edge  $(i, j)$  represents the act of buying a car in year  $i$  and selling in year  $j$ . The weight is the price difference plus the maintenance cost.

$$c(i, j) = \$12,000 - s_{(i-j)} + m_1 + m_2 + \dots + m_{(i-j)}$$

# Car Replacement Problem

- Nodes = {1, 2, 3, 4, 5}
- Edge  $(i, j)$  represents the act of buying a car in year  $i$  and selling in year  $j$ . The weight is the price difference plus the maintenance cost.

$$c(i, j) = \$12,000 - s_{(i-j)} + m_1 + m_2 + \dots + m_{(i-j)}$$

**Selling after 1 year of use**

$$c(0,1) = \$12,000 - 7,000 + 2,000$$



# Car Replacement Problem

- Nodes = {1, 2, 3, 4, 5}
- Edge  $(i, j)$  represents the act of buying a car in year  $i$  and selling in year  $j$ . The weight is the price difference plus the maintenance cost.

$$c(i, j) = \$12,000 - s_{(i-j)} + m_1 + m_2 + \dots + m_{(i-j)}$$

**Selling after 1 year of use**

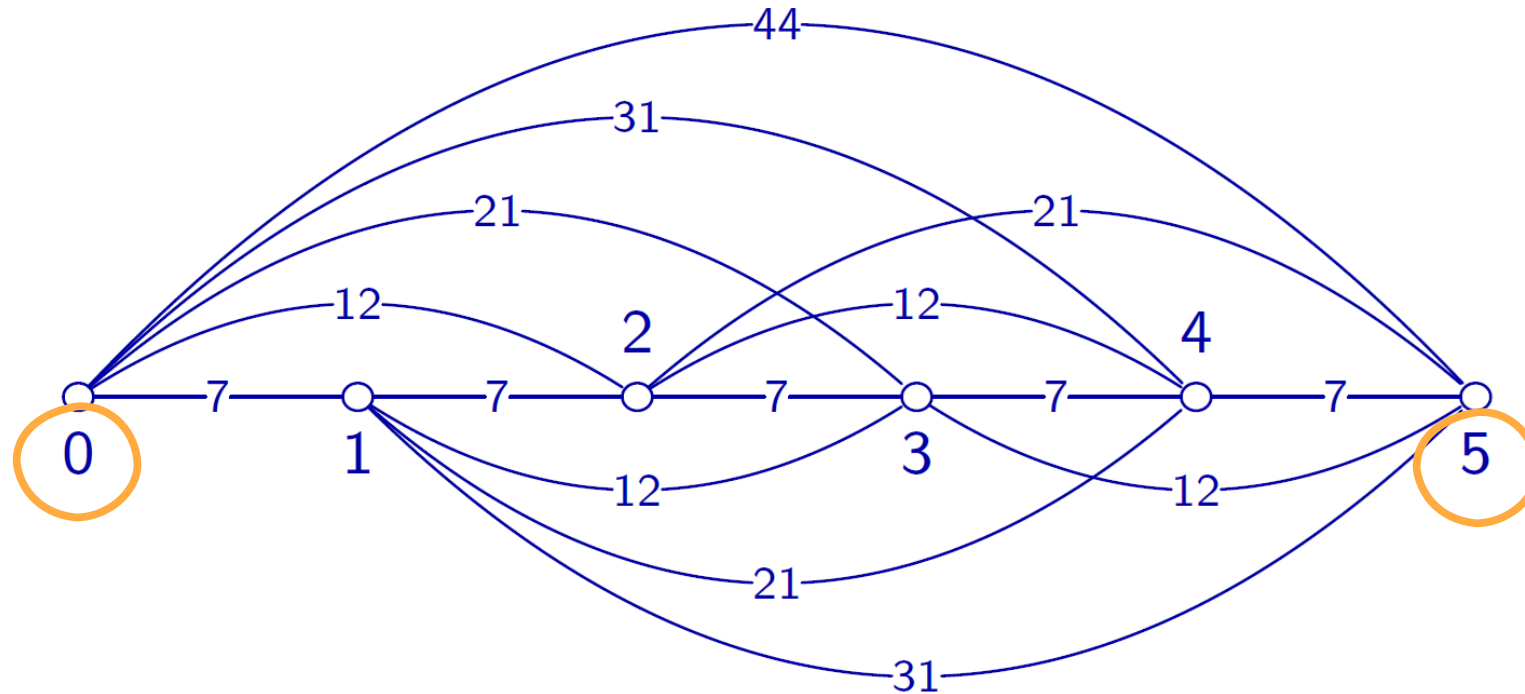
$$c(0,1) = \$12,000 - 7,000 + 2,000$$

**Selling after 2 years of use**

$$c(0,2) = \$12,000 - 6,000 + 2,000 + 4,000$$

# Car Replacement Problem

Now, we can represent this problem as a network, and solve it as the shortest path problem!



---

# Dijkstra's Algorithm

# Dijkstra's algorithm

---

- This algorithm is used to compute a shortest path from a given node  $s$  to every other node (not only  $t$ )

**!** In this case edge-weights are assumed to be non-negative,  
• else the algorithm may output an incorrect answer

# Dijkstra's algorithm

- This algorithm is used to compute a shortest path from a given node  $s$  to every other node (not only  $t$ )

! In this case edge-weights are assumed to be non-negative,  
• else the algorithm may output an incorrect answer

## *In a nutshell:*



It finds the shortest path between two nodes in a graph by iteratively selecting the closest unvisited node and updating the distances of its neighbors.

# Dijkstra's algorithm

---

**Step 1:** Define  $d_u$  for each  $u \in V$ , an estimate on the distance from  $s$  to  $u$

**Step 2:** Initialize  $d_u = \begin{cases} 0, & \text{if } s = u \\ \infty, & \text{otherwise} \end{cases}$

# Dijkstra's algorithm

**Step 1:** Define  $d_u$  for each  $u \in V$ , an estimate on the distance from  $s$  to  $u$

**Step 2:** Initialize  $d_u = \begin{cases} 0, & \text{if } s = u \\ \infty, & \text{otherwise} \end{cases}$

All vertices are initially unprocessed,  
therefore:

|         | $d_u$    |
|---------|----------|
| $u = s$ | 0        |
| 1       | $\infty$ |
| 2       | $\infty$ |
| 3       | $\infty$ |
| ...     | ...      |

# Dijkstra's algorithm

---

**Step 3:** Find an unprocessed vertex  $u$  with smallest  $d_u$

**Step 4:** For each  $(u, v) \in E$ , update  $d_v = \min \{d_v, d_u + C_{uv}\}$

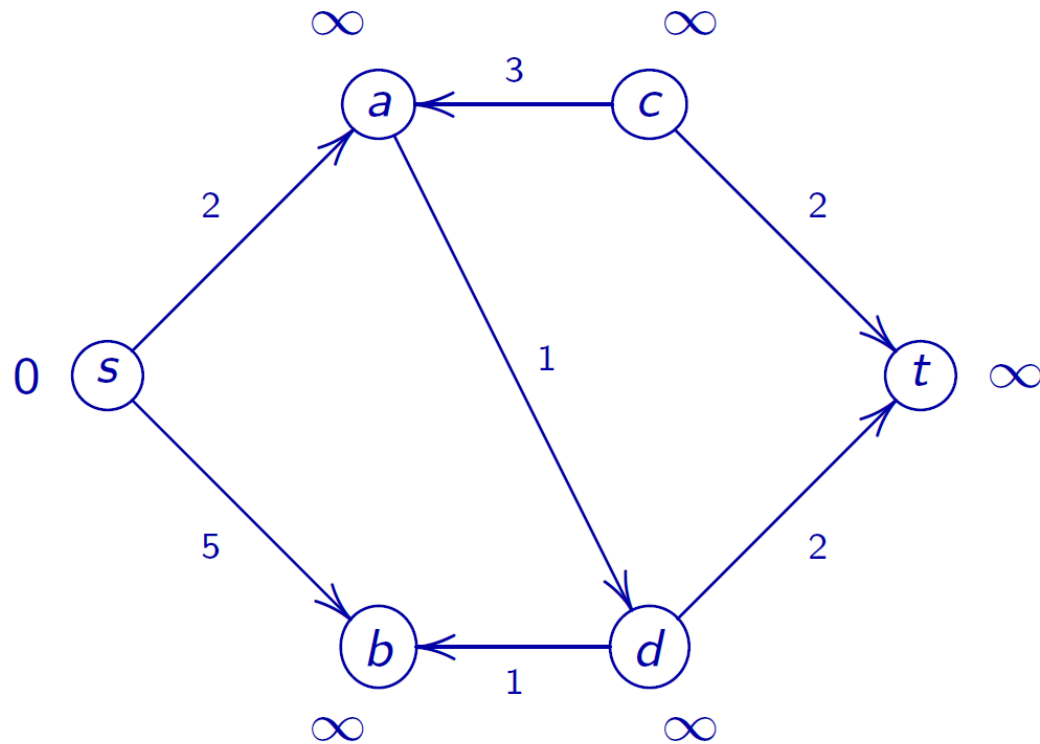
**Step 5:** Mark  $u$  as processed; repeat until all vertices are processed

**Step 6:** Report  $d_t$  as distance from  $s$  to  $t$

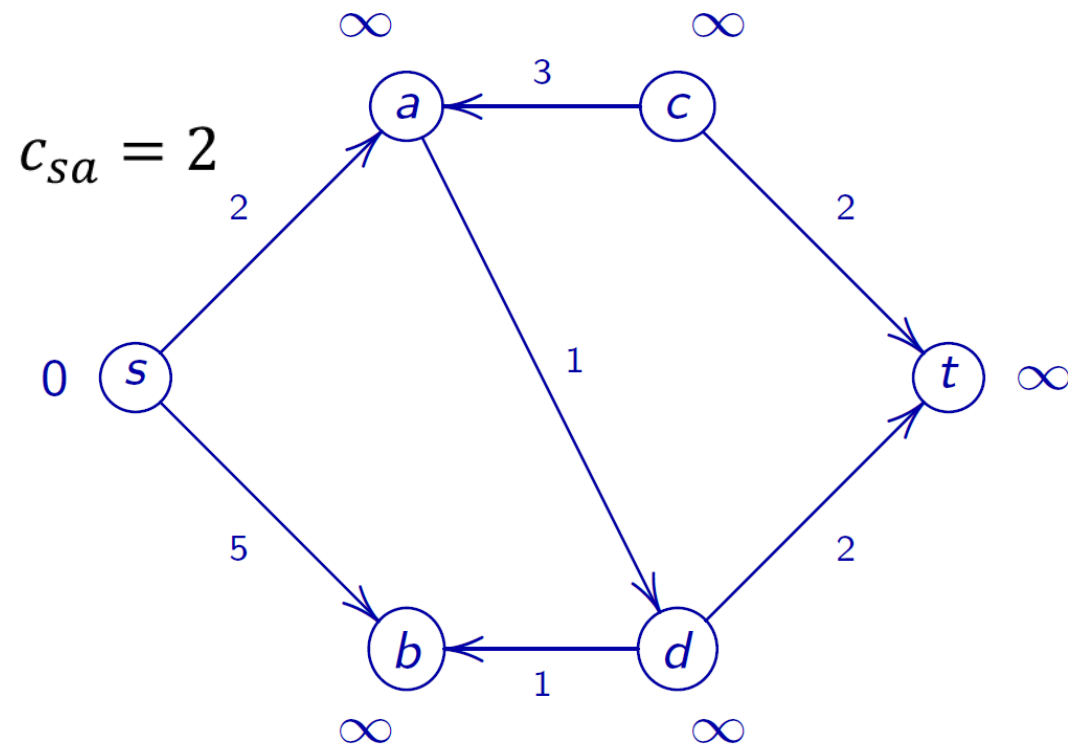


# Example: Dijkstra's algorithm

Initialize:  $d_s = 0, d_a = \infty, d_b = \infty \dots$



# Example: Dijkstra's algorithm

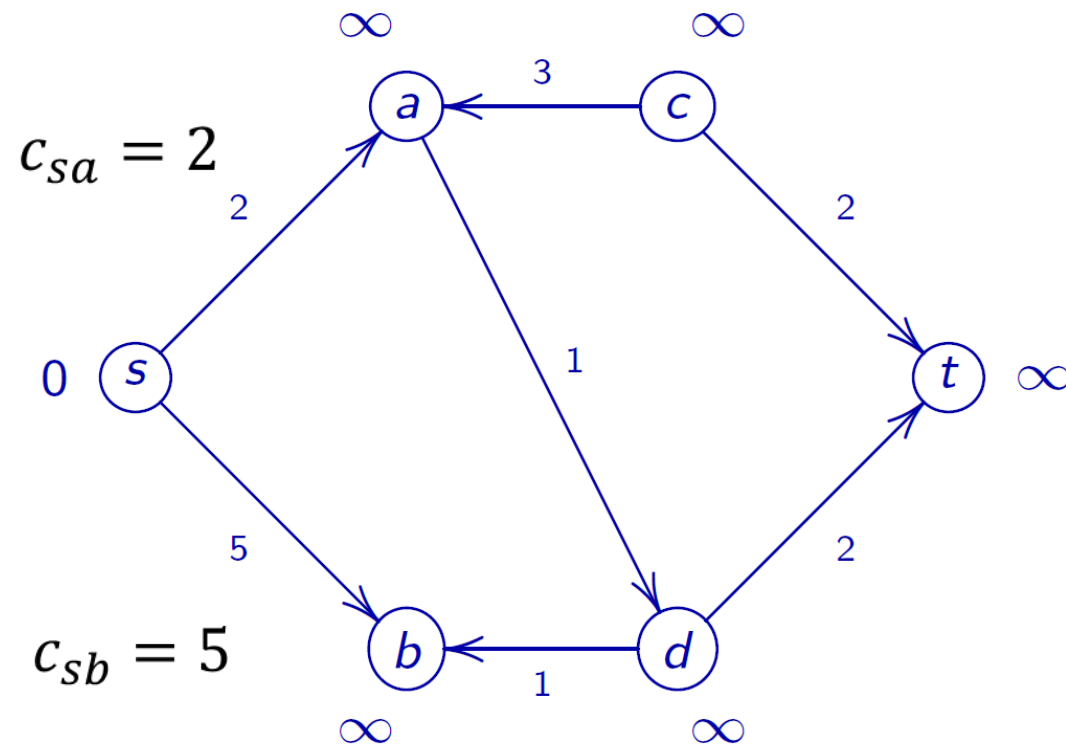


Initialize:  $d_s = 0, d_a = \infty, d_b = \infty \dots$

Then, check  $d_a = \infty >? d_s + c_{sa}$ ,

and update  $d_a = 0 + 2 = 2$

# Example: Dijkstra's algorithm



Initialize:  $d_s = 0, d_a = \infty, d_b = \infty \dots$

Then, check  $d_a = \infty >? d_s + c_{sa}$ ,

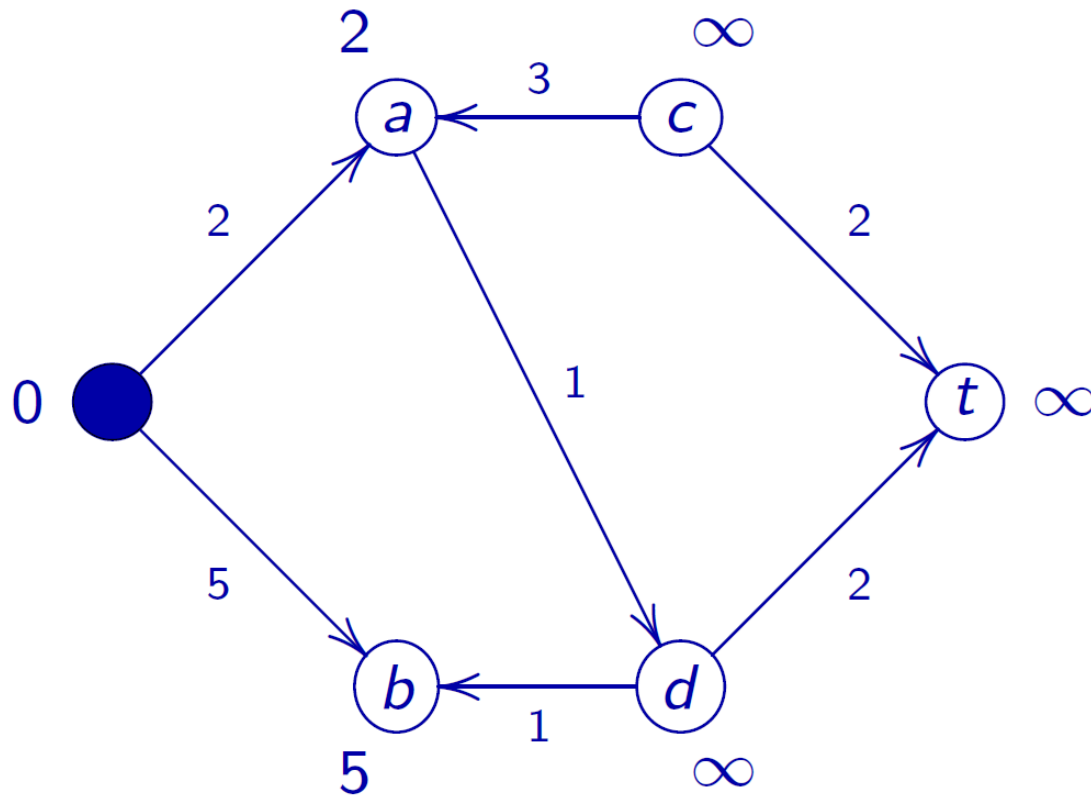
and update  $d_a = 0 + 2 = 2$

Do the same thing for all the unvisited nodes!

$d_b = \infty >? d_s + c_{sb}$ ,

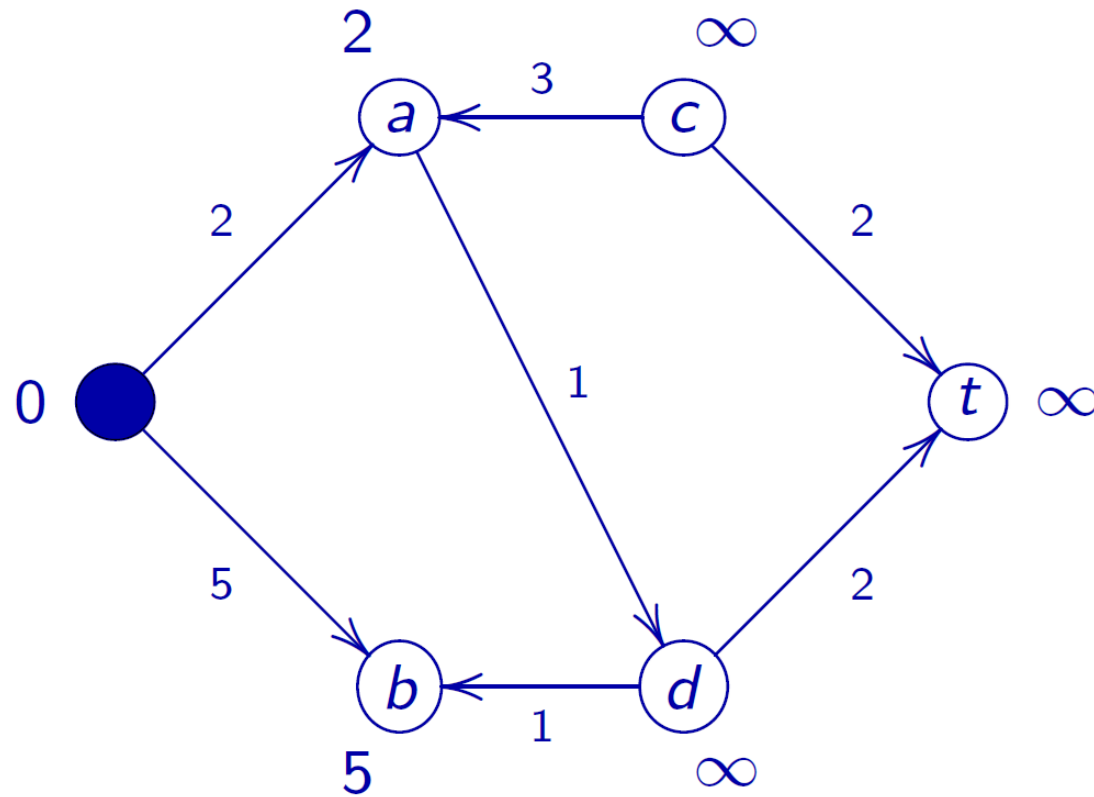
and update  $d_b = 0 + 5 = 5 \dots$

# Example: Dijkstra's algorithm

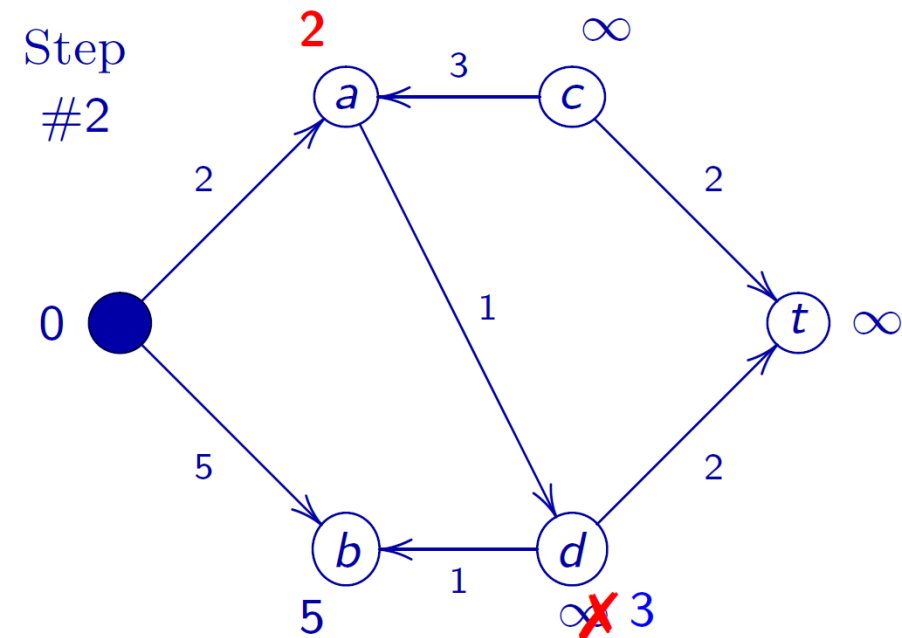


Now,  $d_d = \min\{d_d, d_a + c_{ad}\}$  ,  
and update  $d_d = 2 + 1 = 3$

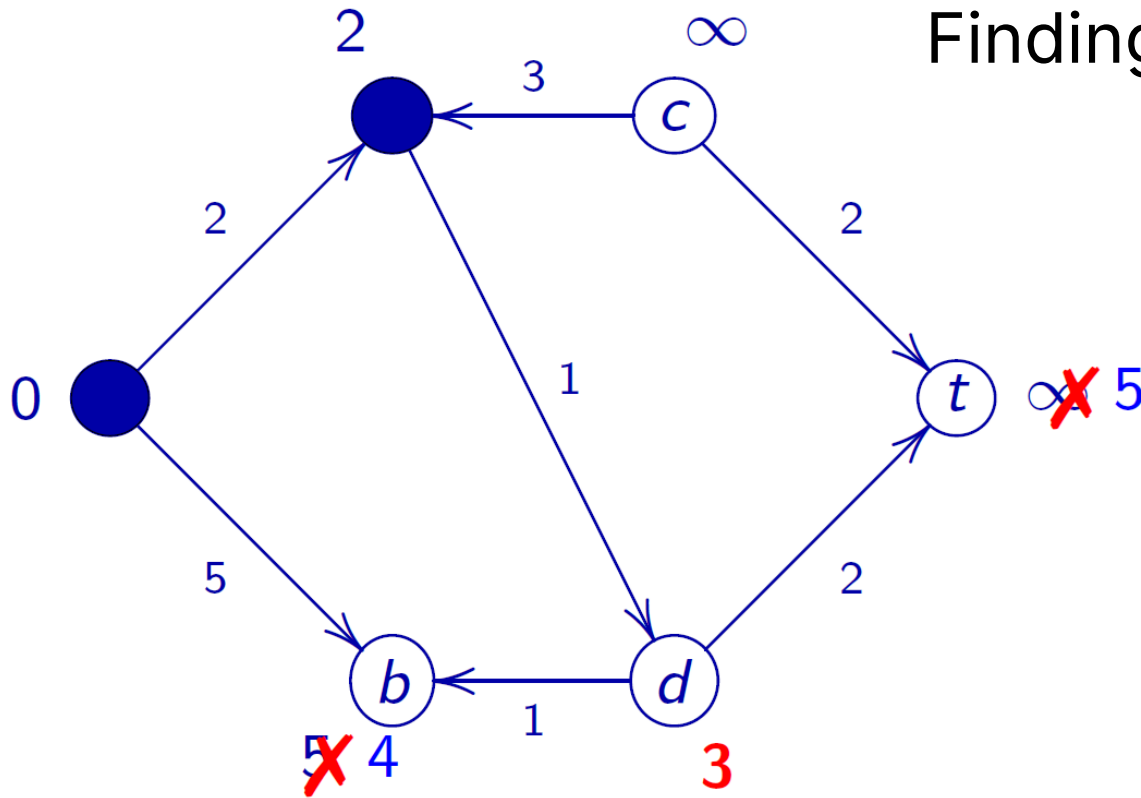
# Example: Dijkstra's algorithm



Now,  $d_d = \min\{d_d, d_a + c_{ad}\}$ ,  
and update  $d_d = 2 + 1 = 3$



# Example: Dijkstra's algorithm

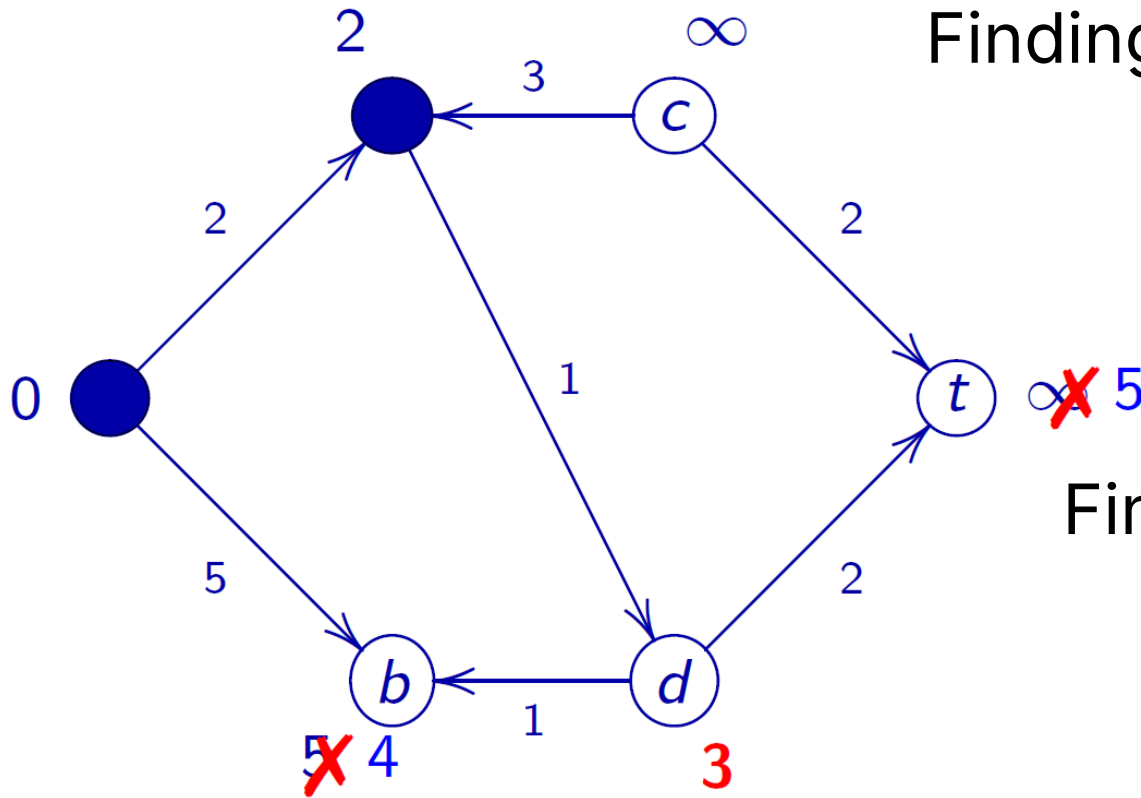


Finding  $d_d$  affects  $d_b$ , as

$$d_b = \min\{d_b, d_d + c_{bd}\},$$

$$d_b = 3 + 1 = 4$$

# Example: Dijkstra's algorithm



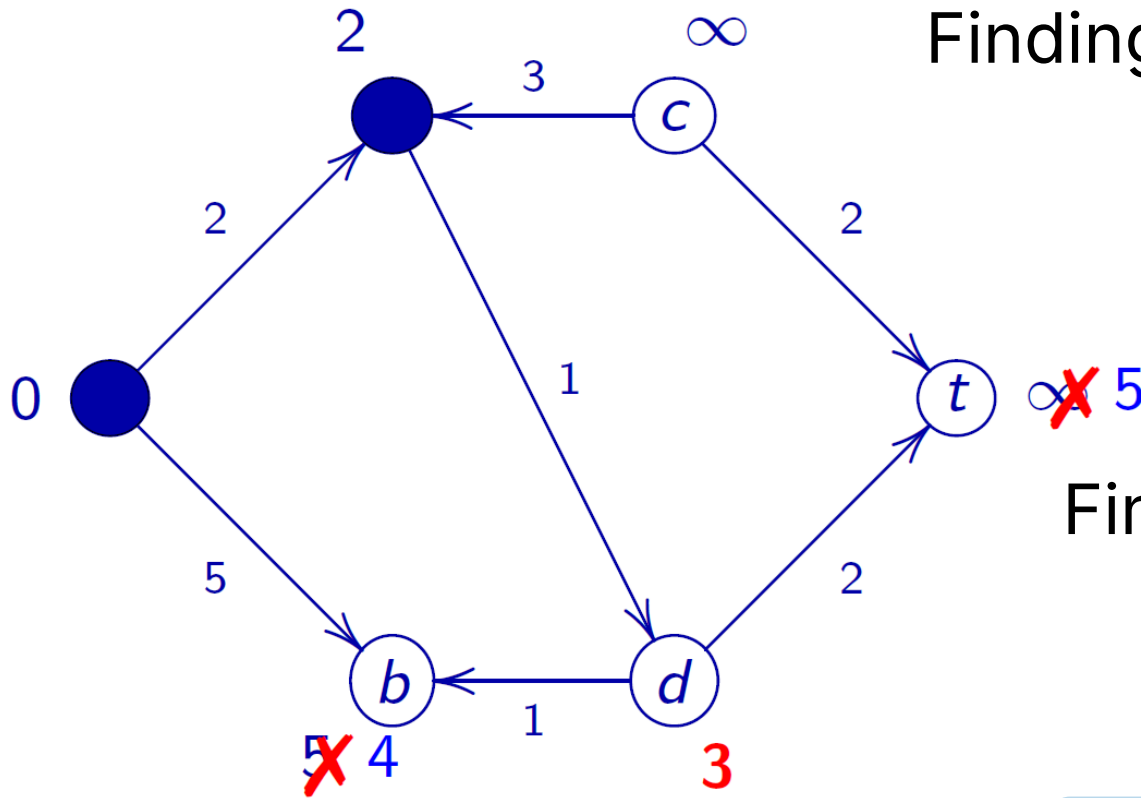
Finding  $d_d$  affects  $d_b$ , as

$$d_b = \min\{d_b, d_d + c_{bd}\},$$
$$d_b = 3 + 1 = 4$$

Finally, let's find  $d_t$ :

$$d_t = \min\{d_t, d_d + c_{dt}\},$$
$$d_t = 3 + 2 = 5$$

# Example: Dijkstra's algorithm



Finding  $d_d$  affects  $d_b$ , as

$$d_b = \min\{d_b, d_d + c_{bd}\},$$
$$d_b = 3 + 1 = 4$$

Finally, let's find  $d_t$ :

$$d_t = \min\{d_t, d_d + c_{dt}\},$$
$$d_t = 3 + 2 = 5$$

**Shortest path from s to t is 5!**



# Example: Dijkstra's algorithm

This method can also be shown in table format:

| Step # | <i>s</i> | <i>a</i>      | <i>b</i>      | <i>c</i>                   | <i>d</i>      | <i>t</i>      |
|--------|----------|---------------|---------------|----------------------------|---------------|---------------|
| 1.     | <u>0</u> | $\infty$<br>2 | $\infty$<br>5 | $\infty$                   | $\infty$      | $\infty$      |
| 2.     | 0*       | <u>2</u>      | 5             | $\infty$                   | $\infty$<br>3 | $\infty$      |
| 3.     | 0*       | 2*            | 5<br>4        | $\infty$                   | <u>3</u>      | $\infty$<br>5 |
| 4.     | 0*       | 2*            | <u>4</u>      | $\infty$                   | 3*            | 5             |
| 5.     | 0*       | 2*            | 4*            | $\infty$                   | 3*            | <u>5</u>      |
| 6.     | 0*       | 2*            | 4*            | <u><math>\infty</math></u> | 3*            | 5*            |
| final  | 0*       | 2*            | 4*            | $\infty^*$                 | 3*            | 5*            |

# Example: Project Management Problem

---

## Background:

A company wants to introduce a new model of its ever-popular cell phone to the market. To gauge the demand, a limited batch of product is to be brought to the market. The product is assembled from two parts, a circuit board, and housing. In addition, workers need to be trained and raw material procured.

# Example: Project Management Problem

---

After the initial assessment, the project manager put together a list of all tasks along with estimated duration of each task. The tasks are interdependent, some need to finish before others can start.

First, (A) workers need to be trained and (B) raw materials purchased which takes 6 and 9 days, respectively. After these tasks are finished, both parts of the product, (C) the circuit board and (D) the housing, are manufactured taking 7 and 8 days, respectively. Each circuit board, once manufactured, needs to undergo additional testing (E) to comply with FCC regulation which takes 10 days. Afterwards, the cell phone is assembled, packaged, and shipped (F) which takes 12 days.

# Example: Project Management Problem

---

## Question:

What is the minimum number of days before the product can reach the market? What is the critical path of the production, i.e., the sequence of tasks delaying any of which delays the whole production?

# Example: Project Management Problem

---

We identify main checkpoints:

- (1) start, (2) completion of tasks A and B,
- (3) completion of task D, (4) completion of tasks C, E, and
- (5) finish

Nodes {1, 2, 3, 4, 5}

# Example: Project Management Problem

We identify main checkpoints:

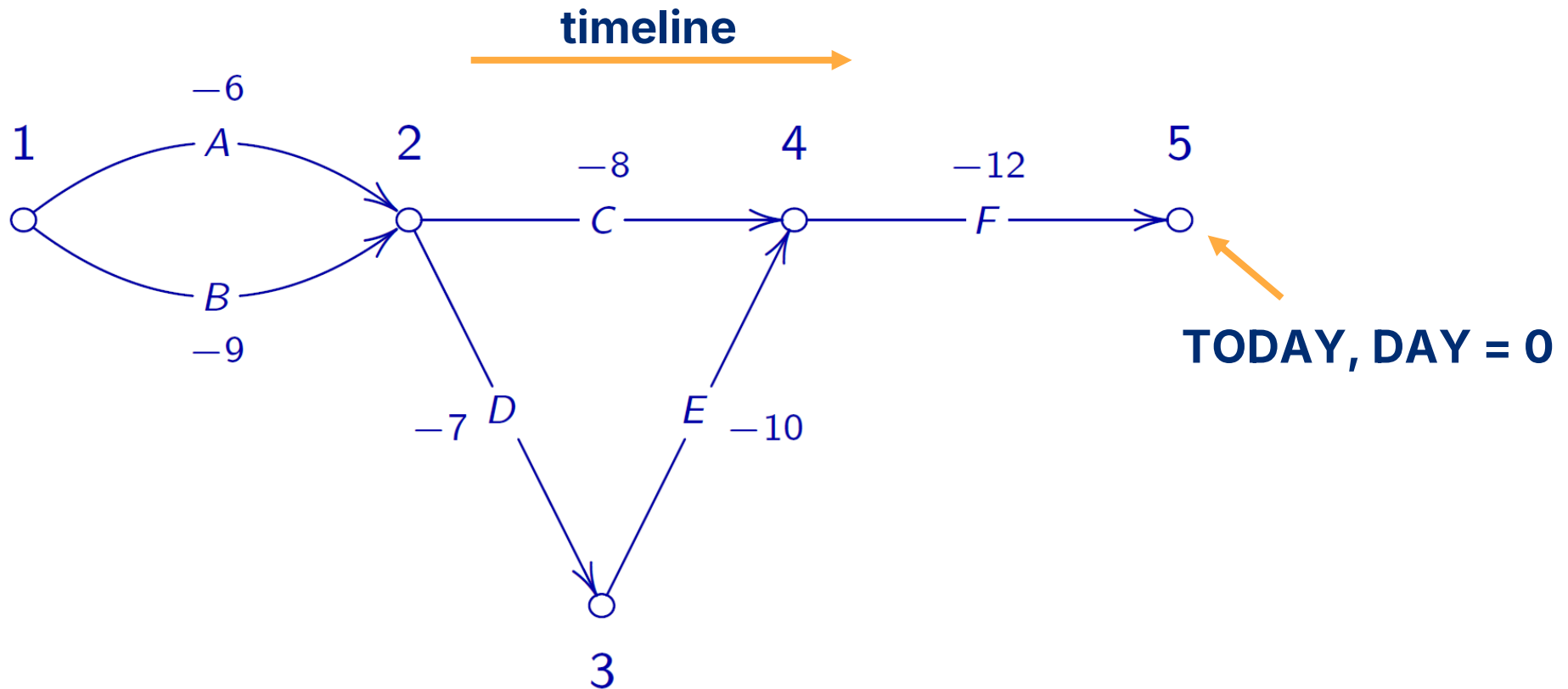
- (1) start, (2) completion of tasks A and B,
- (3) completion of task D, (4) completion of tasks C, E, and
- (5) finish

Nodes {1, 2, 3, 4, 5}

- ! Edges correspond to tasks that connect checkpoints, weights are durations with negative sign.

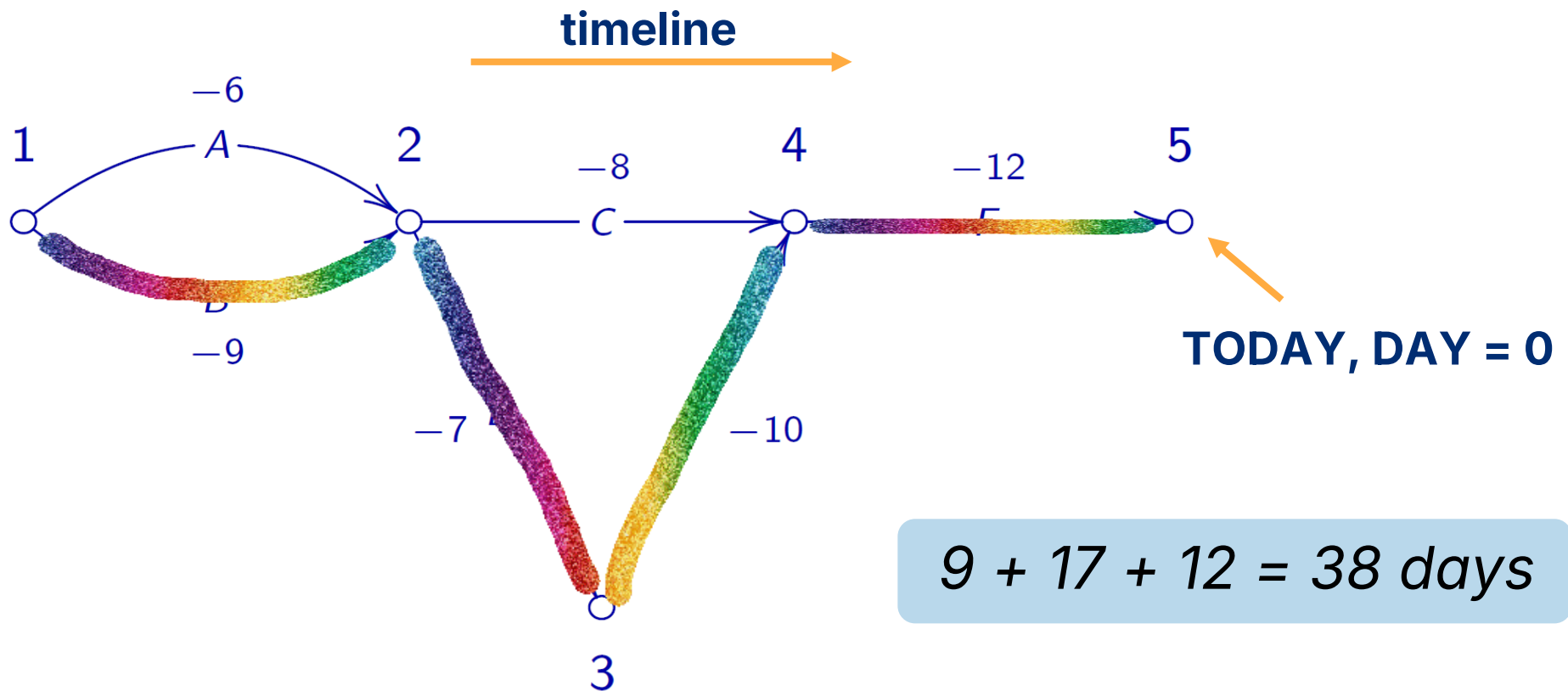
# Example: Project Management Problem

Represent this in the network form



# Example: Project Management Problem

Represent this in the network form





# Announcement

---

- On **Wednesday, March 25<sup>th</sup>** we will have an in-class assignment that will:
  - Involve programming
  - Focus on the network models covered so far
  - Take the whole class time
  - Be completed individually & submitted during the class
- **Notes:**
  - No AI tools will be used
  - Attendance will be taken
  - This will impact your final course grade as **Assignment 4**